CS4048 — DATA SCIENCE

Lecture 12

October 02, 2025

Fall 2025

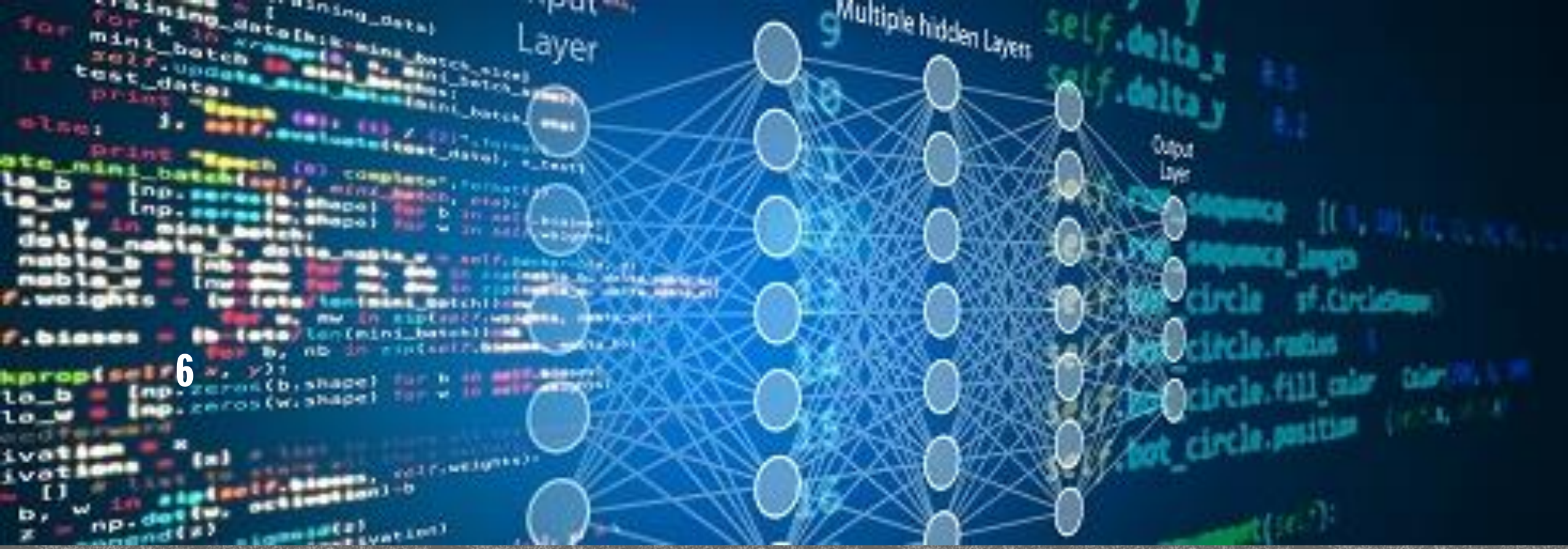
FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Data Wrangling/Cleaning
  - Data visualization
  - Data normalization
  - Data transformation
- Exploratory Data Analysis: descriptive statistics



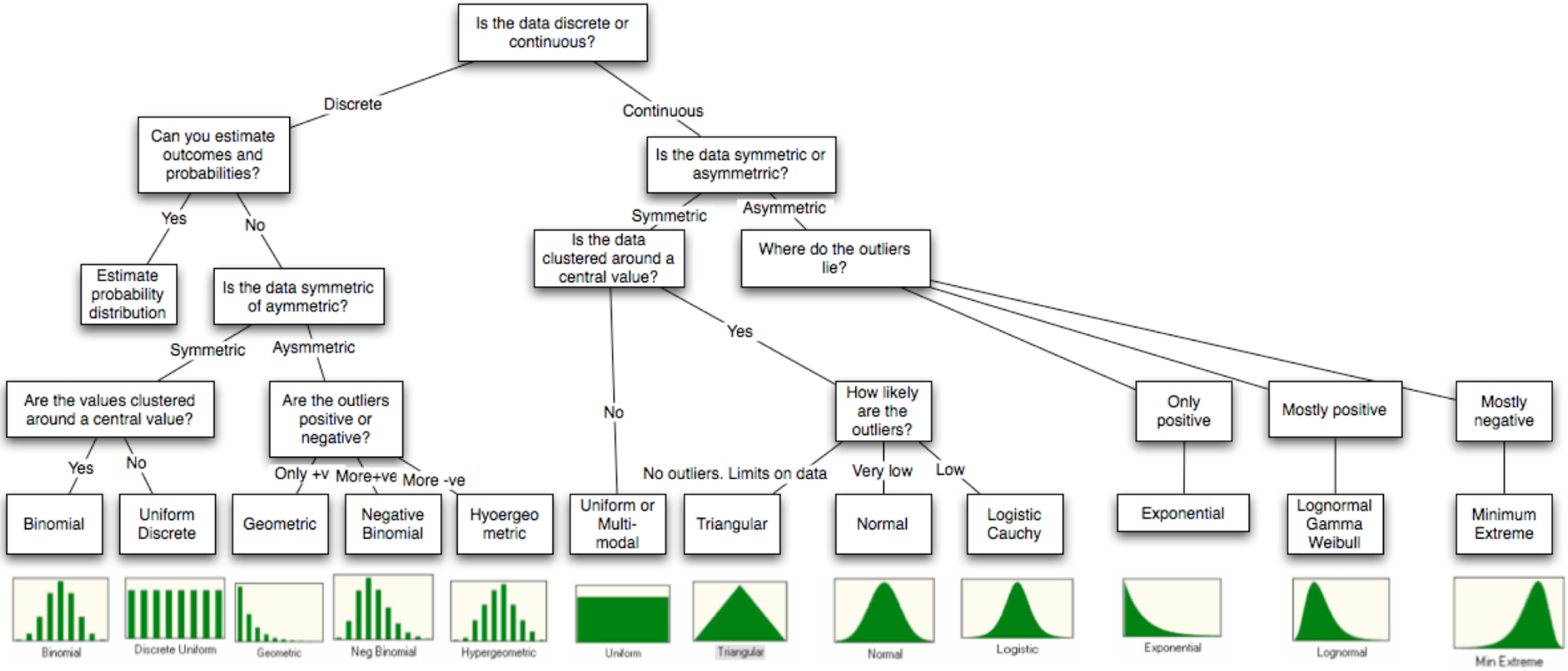
DATA TRANSFORMATION

DATA TRANSFORMATION

- A feature transformation may be needed because:
  - The value codings might not be useful for analysis;
  - we may want to apply a mathematical function to a feature; or
  - we might want to pull information out of a feature and create a new feature

- Three basic kinds of transformations:
  - type conversions,
  - mathematical transformations, and
  - extractions

Figure 6A.15: Distributional Choices



1. TRANSFORMATION: TYPE CONVERSION

- This kind of transformation occurs when we convert the data from one format to another to make the data more useful for analysis.
- We might convert information stored as a string to another format.
- Timestamps come in many different formats!

- Examples:
  - Convert price stored as strings "$2.17" to the number 2.17
  - Convert a time stored as a string, such as "1955-10-12", to a pandas Timestamp object

TRANSFORMING TIMESTAMPS

- A timestamp is a data value that records a specific date and time.
- A dataset with different date formats, such as “MM/DD/YYYY” and “YYYY/MM/DD.”

- Examples: Jan 1 2020 2pm or 2021-01-31 14:00:00

TRANSFORMING TIMESTAMPS

- By default, pandas reads in the date column as an integer:

	business_id	score	date	type
0	19	94	20160513	routine
1	19	94	20171211	routine
2	24	98	20171101	routine
3	24	98	20161005	routine

```
insp['date'].dtype
```

```
dtype('int64')
```

- We can convert the date column to the pandas Timestamp storage type and extract the day of the week.

```
date_format = '%Y%m%d'

insp_dates = pd.to_datetime(insp['date'], format=date_format)
insp_dates[:3]
```

```
0   2016-05-13
1   2017-12-11
2   2017-11-01
Name: date, dtype: datetime64[ns]
```

```
insp_dates.dt.year[:3]
```

```
0    2016
1    2017
2    2017
Name: date, dtype: int32
```

TRANSFORMING TIMESTAMPS

- More on timestamp

`pd.to_datetime()`

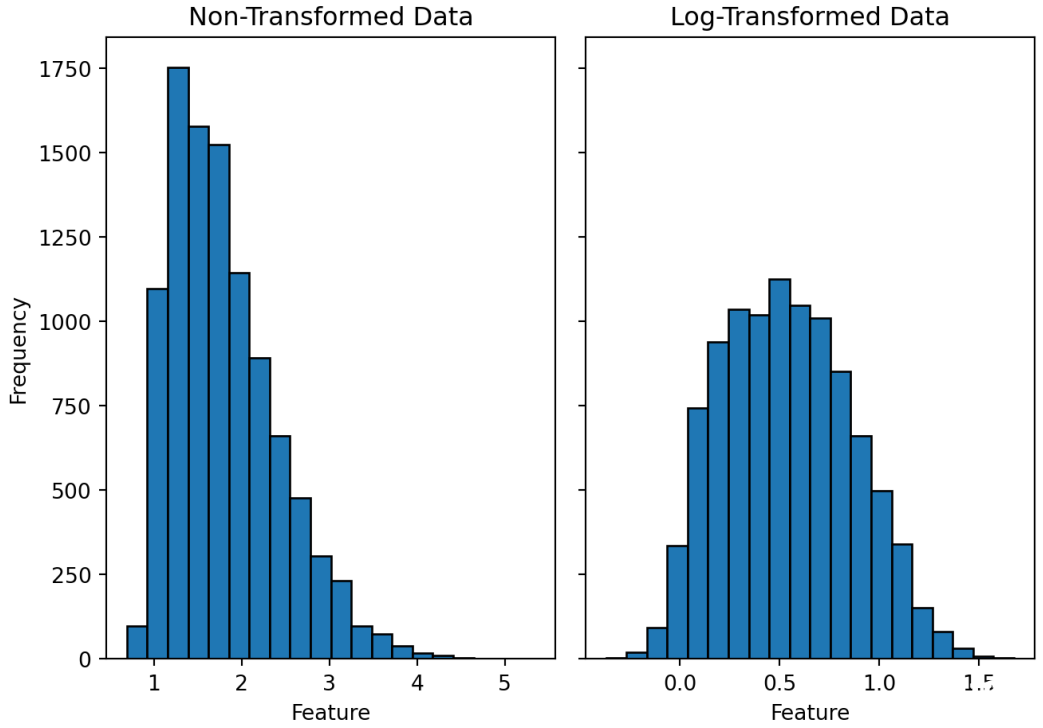
`pd.series.dt.date()`
`pd.series.dt.dayofweek()`
`pd.series.dt.hour()`

2. TRANSFORMATION: MATHEMATICAL

- This kind of transformation occurs when we apply some mathematical function to:
  - Change the units of a measurement (e.g., pounds to kilograms)
  - Make the data distribution symmetric (e.g., logarithmic, square root, exponential transformations)
  - Create a new feature from arithmetic operations (e.g., combine mass & weight to create body mass using height/weight^2

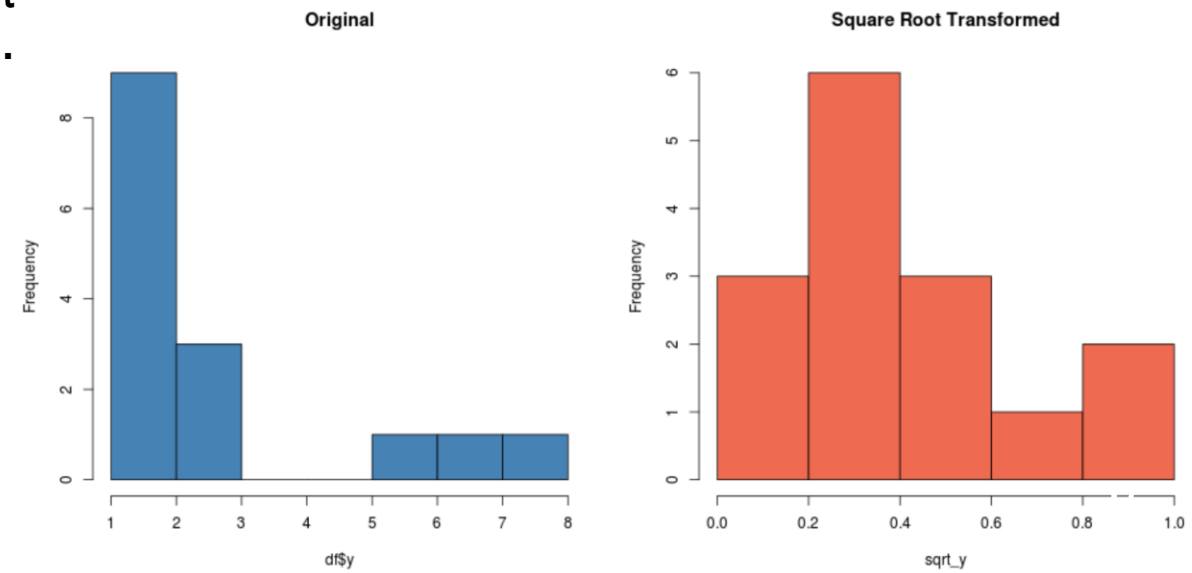
2. TRANSFORMATION: MATHEMATICAL

- **Log transformation:** requires taking the logarithm of the data values.
  - Log transformation is often used when the data is highly skewed, meaning most of the data points fall toward one end of the distribution.
  - By taking the logarithm of the values, the distribution can be shifted toward a more symmetrical shape, making it easier to analyze.



2. TRANSFORMATION: MATHEMATICAL

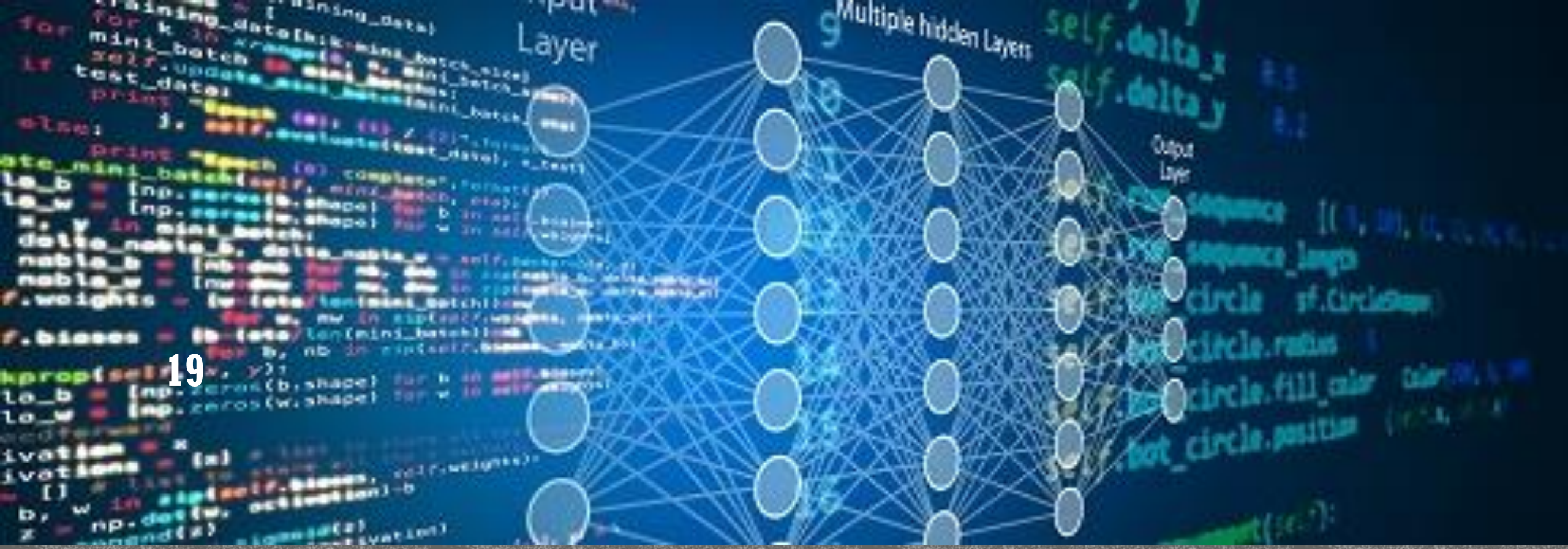
- **Square root transformation:** taking the square root of the data values.
  - Similar to log transformation, square root transformation is often used to address issues with skewness and make the data more normally distributed.
  - Square root transformation is also useful when the data contains values close to zero, as taking the square root of these values can bring them closer to the rest of the data and reduce the impact of extreme values.



TRANSFORMATION: EXTRACTION

- Create a feature by extraction, where the new feature contains partial information taken from another feature.

- Not the scope of this course!



DATA NORMALIZATION

DATA NORMALIZATION

- The process of transforming features in a dataset to a common scale, without distorting differences in the ranges of values.
- This is important when your data has varying scales or units, as some machine learning algorithms are sensitive to the magnitude of the data, such as distance-based models (e.g., k-NN, SVM) and gradient-based algorithms (e.g., gradient descent).

- Types of normalization:
  1. Min-Max Normalization (also known as Feature Scaling)
  2. Z-Score Normalization (also known as Standardization)
  3. Mean-based scaling
  4. Robust Scaling
  5. Unit Vector Scaling

1. MIN-MAX NORMALIZATION

- Min-Max normalization transforms the features to a fixed range, usually [0, 1], by scaling the data based on the minimum and maximum values in the dataset.

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Where:

- X = original value
- X_{min} = minimum value of the feature
- X_{max} = maximum value of the feature
- X_{norm} = normalized value

EXAMPLE

- Min-max normalization values typically range between 0 and 1

$$v' = \frac{v - \min(v)}{\text{range}(v)} = \frac{v - \min(v)}{\max(v) - \min(v)}$$

- Example: Let income range \$12,000 to \$98,000.Then \$73,600 is mapped to:

$$v' = \frac{73600 - 12000}{98000 - 12000} = \frac{61600}{86000} = 0.716$$

1. MIN-MAX NORMALIZATION

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Sample data
data = np.array([[10], [20], [30], [40], [50]])

# Create a MinMaxScaler object
scaler = MinMaxScaler()

# Fit and transform the data
normalized_data = scaler.fit_transform(data)
print("Min-Max Normalized Data:")
print(normalized_data)
```

NORMALIZATION (OR STANDARDIZATION)

- Min-max normalization to [new_minA, new_maxA]

$$v' = \frac{v - min_A}{max_A - min_A}(new\_max_A - new\_min_A) + new\_min_A$$

- Example: Let income range \$12,000 to \$98,000 normalized to [1.0, 5.0]. Then \$73,600 is mapped to:

$$v' = \frac{73600 - 12000}{98000 - 12000}(5.0 - 1.0) + 1.0 = \frac{61600}{86000}(4) + 1 = 3.864$$

2. Z-SCORE NORMALIZATION

- Z-score normalization (or Standardization) transforms the data to have a mean of 0 and a standard deviation of 1.

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = original value
- μ = mean of the feature
- σ = standard deviation of the feature
- Z = standardized value

EXAMPLE

- Z-score standardization: (μ: mean, σ: standard deviation):

$$v' = \frac{v - \mu_A}{\sigma_A}$$

- Example: Let $\mu = 54{,}000$, $\sigma = 16{,}000$. Then:

$$\frac{73{,}600 - 54{,}000}{16{,}000} = 1.225$$

Data values that lie below mean will have a negative z-score standardization.
Data values falling exactly on the mean will have a zero z-score standardization.
Data values that lie above the mean will have a positive z-score standardization.

2. Z-SCORE NORMALIZATION

- Z-score standardization: (μ: mean, σ: standard deviation):

$$v' = \frac{v - \mu_A}{\sigma_A}$$

- Z-score methods states that a data value is an outlier if it has a Z-score that is either less than -3 or greater than 3.

Caveat: when choosing a method for evaluating outliers, it may not seem appropriate to use measures that are themselves sensitive to their presence.

2. Z-SCORE NORMALIZATION

```
from sklearn.preprocessing import StandardScaler


# Sample data
data = np.array([[10], [20], [30], [40], [50]])


# Create a StandardScaler object
scaler = StandardScaler()


# Fit and transform the data
standardized_data = scaler.fit_transform(data)


print("Z-Score Normalized Data:")
print(standardized_data)
```

4. ROBUST SCALING

- It scales the data by using the median and the interquartile range (IQR) instead of the mean and standard deviation, making it robust to outliers.

$$X_{\text{robust}} = \frac{X - \text{Median}}{\text{IQR}}$$

Where:

- **Median** = the middle value of the feature
- IQR = interquartile range ($Q3 - Q1$) of the feature

- This method is especially useful when your dataset has many outliers.

4. ROBUST SCALING

```
from sklearn.preprocessing import RobustScaler


# Sample data with an outlier
data = np.array([[10], [20], [30], [40], [1000]])


# Create a RobustScaler object
scaler = RobustScaler()


# Fit and transform the data
robust_scaled_data = scaler.fit_transform(data)


print("Robust Scaled Data:")
print(robust_scaled_data)
```

5. UNIT VECTOR SCALING

- Unit vector scaling (also known as **Vector Normalization**) transforms the data such that the magnitude (or Euclidean norm) of each data point becomes 1.
- This is useful when you want to normalize a dataset for certain distance-based models, like k-NN or when features have different units of measurement.

The formula for Unit Vector Scaling is:

$$X_{\text{unit}} = \frac{X}{\|X\|}$$

Where $\|X\|$ is the **Euclidean norm** (magnitude) of the vector X, which is calculated as:

$$\|X\| = \sqrt{X_1^2 + X_2^2 + \cdots + X_n^2}$$

5. UNIT VECTOR SCALING

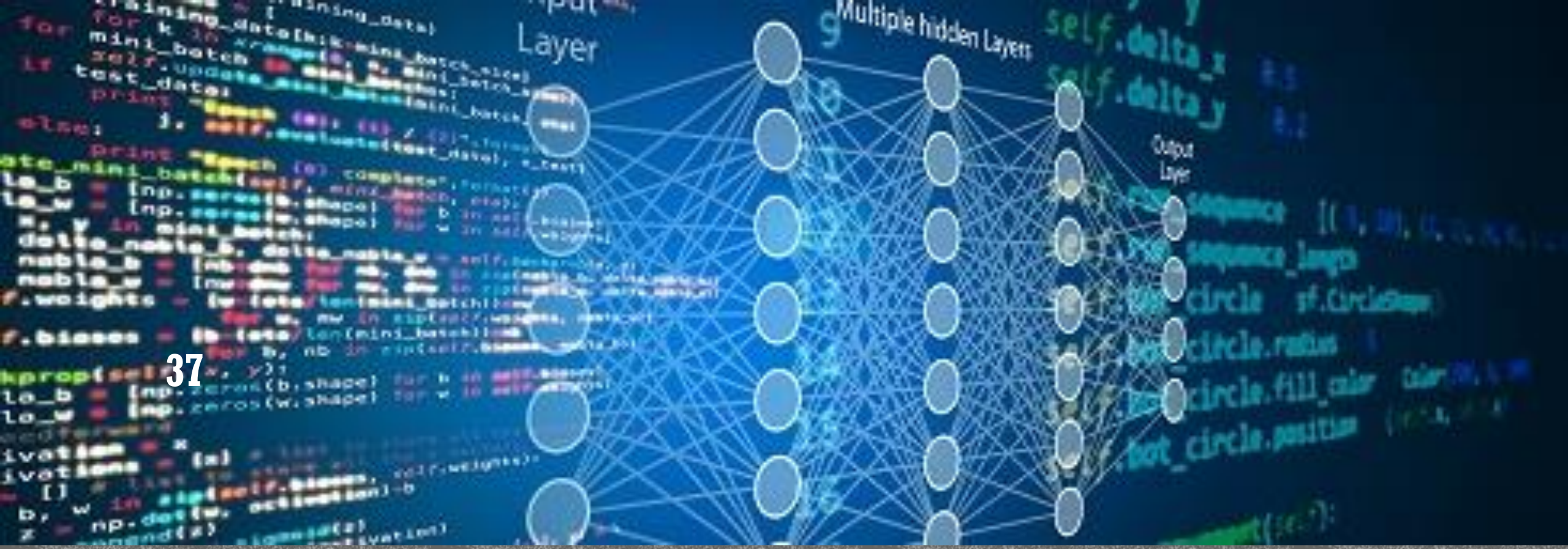
```
from sklearn.preprocessing import Normalizer


# Sample data
data = np.array([[10], [20], [30]])


# Create a Normalizer object (default is L2 norm, which is Unit Vector Scaling)
scaler = Normalizer()


# Fit and transform the data
unit_vector_scaled_data = scaler.fit_transform(data)


print("Unit Vector Scaled Data:")
print(unit_vector_scaled_data)
```

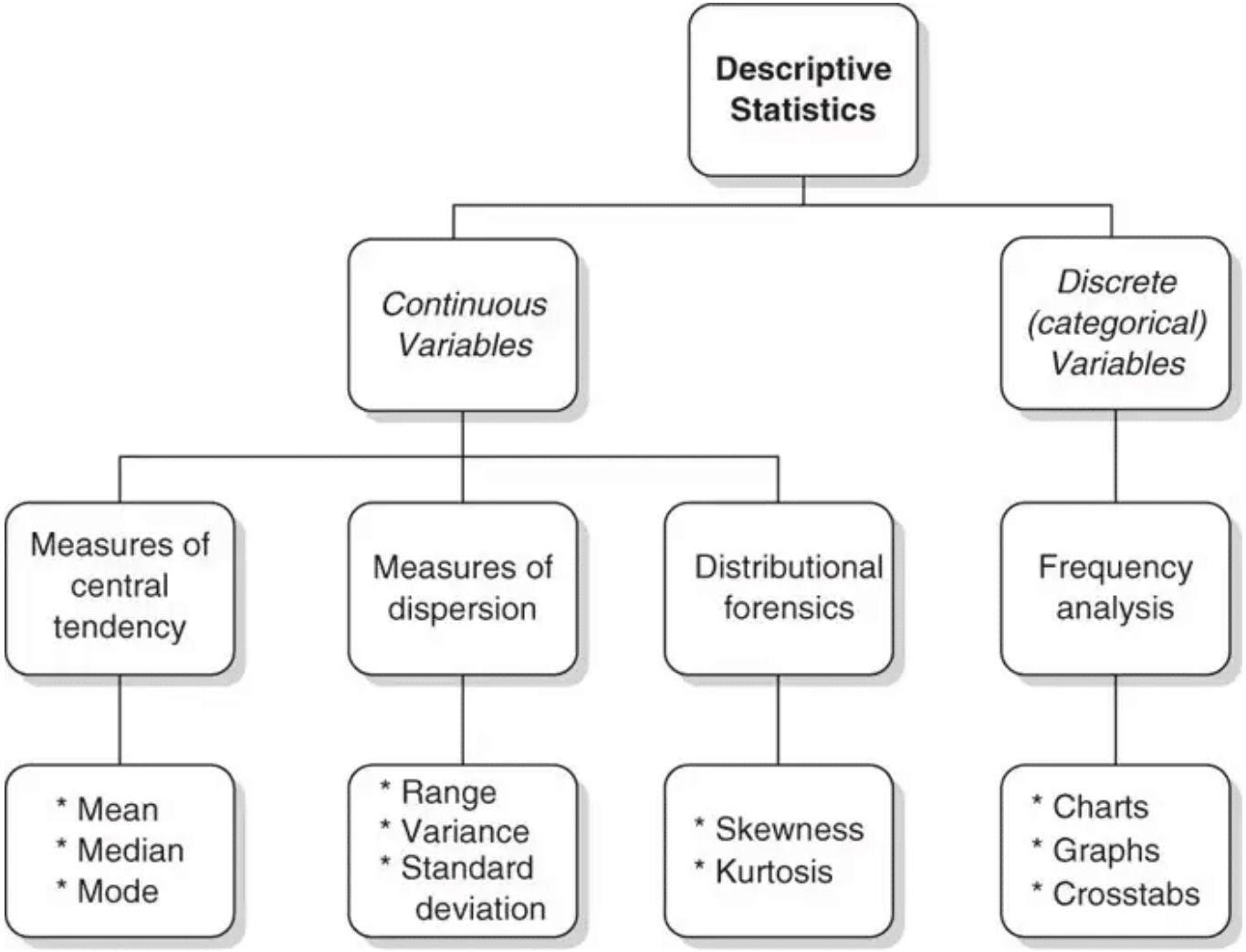


EDA: DESCRIPTIVE STATISTICS

EDA: DESCRIPTIVE STATISTICS

John Tukey

Exploratory Data Analysis, 1977



https://i0.wp.com/nariyoo.com/wp-content/uploads/2022/11/image-27.png?resize=700%2C537&ssl=1

FREQUENCIES AND MODE

- Useful for **unordered categorical values**

- **Frequency:** the measure of count with which each value occurs in data

$$frequency(\mathbf{v_i}) = \frac{number\ of\ objects\ with\ attribute\ value\ vi}{m}$$

- **Mode:** value that has the highest frequency

PERCENTILES

- For ordered data, it is more useful to consider the **percentiles** of a set of values.

- Percentile is a measure used in statistics indicating the value below which a given percentage of observations in a group of observations fall.
  - For example, the 50th percentile is the score below which 50% of the observations may be found.

MEASURING CENTRAL TENDENCY

- For continuous data, two most commonly used summary statistics are: mean and median.
- **Mean** (algebraic measure) (sample vs. population): $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ $\mu = \frac{\sum x}{N}$

  Note: n is sample size and N is population size.
  - Weighted arithmetic mean: $\bar{x} = \frac{\sum_{i=1}^{n} w_i x_i}{\sum_{i=1}^{n} w_i}$
  - Trimmed mean: chopping extreme values
- **Median:**
  - Middle value if odd number of values, or average of the middle two values otherwise
  - Estimated by interpolation (for grouped data):

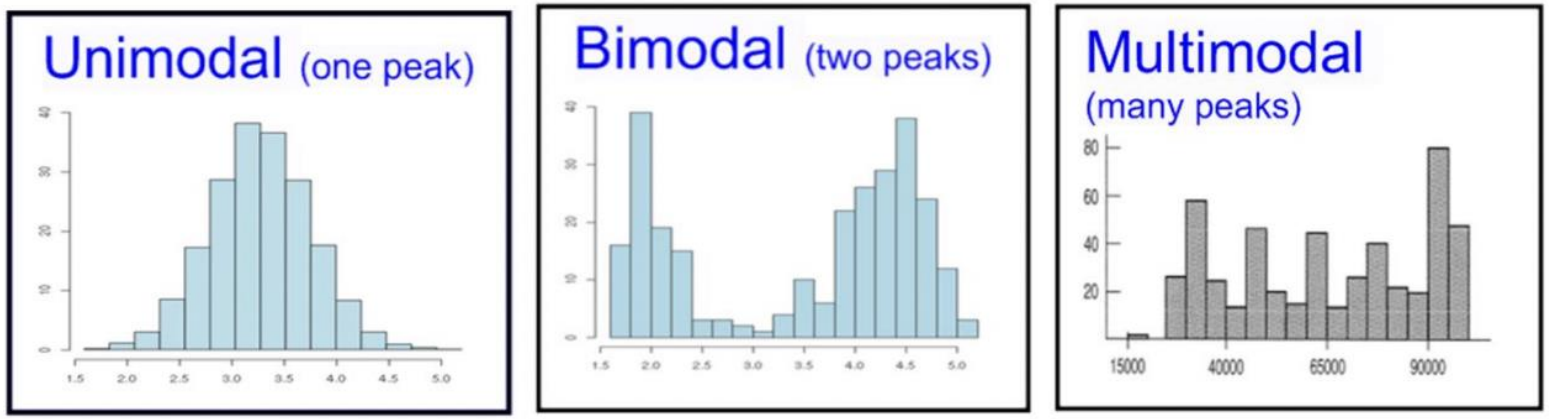
$$median = L_1 + (\frac{n/2 - (\sum freq)l}{freq_{median}})width$$

age	*frequency*
1–5	200
6–15	450
16–20	300
21–50	1500
51–80	700
81–110	44

MEASURING CENTRAL TENDENCY

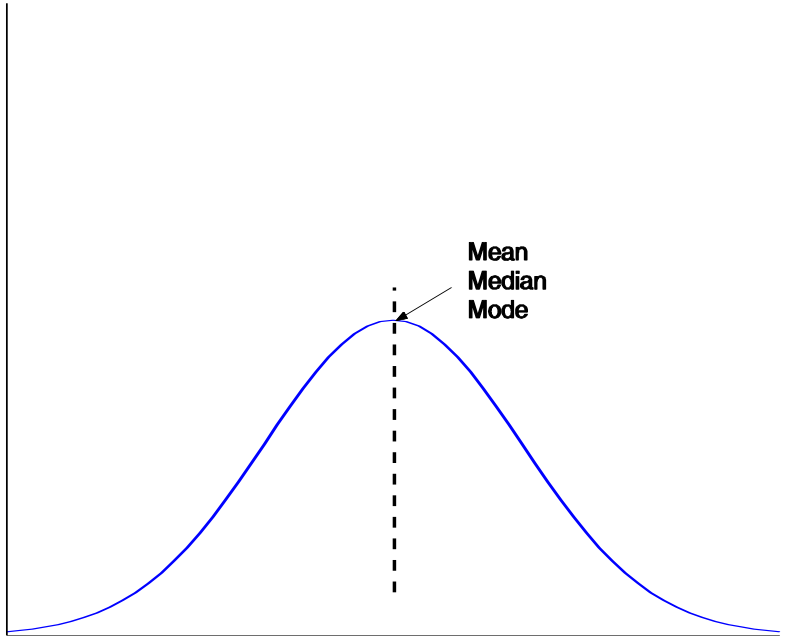
- **Mode** (some may prefer to use mode for continuous data)
  - Value that occurs most frequently in the data
  - Unimodal, bimodal, trimodal
  - Empirical formula:

$$mean - mode = 3 \times (mean - median)$$

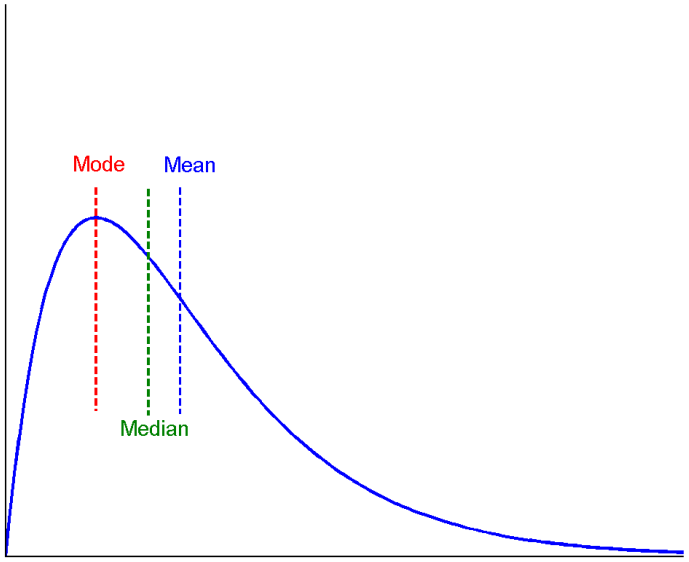


SYMMETRIC VS. SKEWED DATA

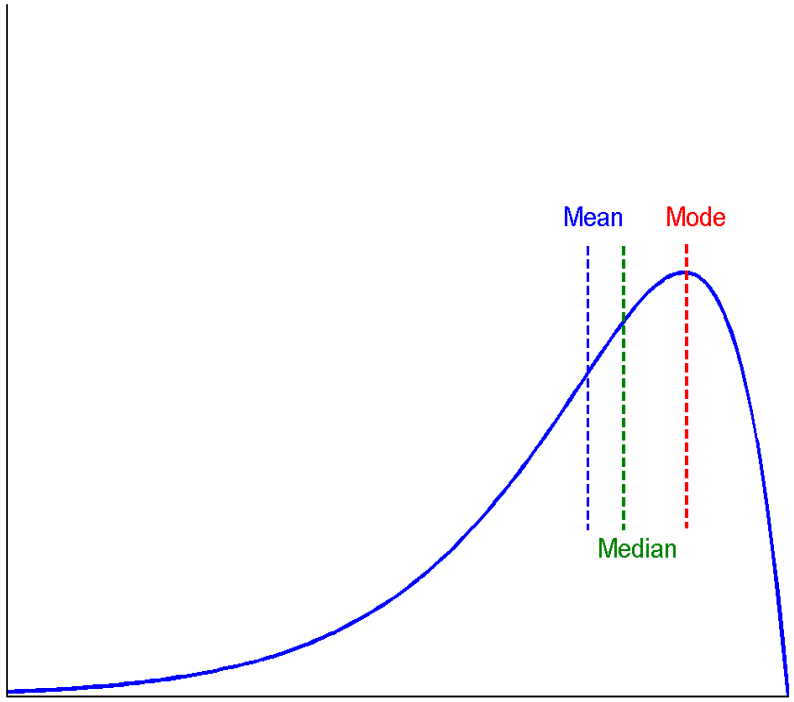
- Median, mean and mode of symmetric, positively and negatively skewed data



Symmetric (not skewed)



Positively (right) skewed



Negatively (left) skewed

MEASURING CENTRAL TENDENCY

TABLE 2.3 The two portfolios have the same mean, median, and mode, but are clearly different

Stock Portfolio A	Stock Portfolio B
1	7
11	8
11	11
11	11
16	13

Mean = 10
Median = 11
Mode = 11

Thus, measures of centre do not provide us with a complete picture.

MEASURING DATA DISPERSION/SPREAD/VARIABILITY

- Another set of commonly used summary statistics for continuous data are those that measure the dispersion or spread of a set of values.

- These indicate if the attribute values are widely spread out or concentrated around the center (or mean).
  - It includes range, variance, standard distribution

MEASURING DATA DISPERSION/SPREAD/VARIABILITY

- **Range:** the simplest measure of spread

$$range(\mathbf{x}) = \max(x) - \min(x)$$

- Variance and standard deviation (sample:s, population:σ)
- **Variance:** (algebraic, scalable computation)

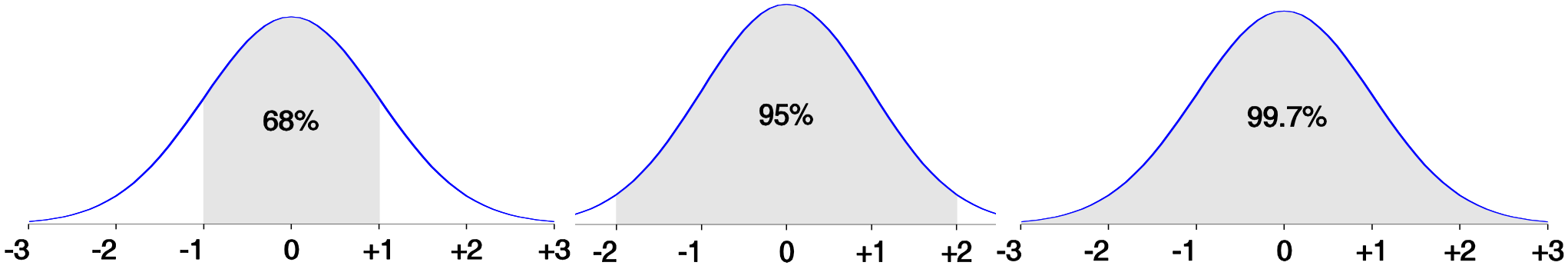
$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 = \frac{1}{n-1}[\sum_{i=1}^{n} x_i^2 - \frac{1}{n}(\sum_{i=1}^{n} x_i)^2]$$

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{n}(x_i - \mu)^2 = \frac{1}{N}\sum_{i=1}^{n} x_i^2 - \mu^2$$

- **Standard deviation (s or σ):** is the square root of variance s^2(or σ^2)
  - Most values lie within two SDs of the mean.

PROPERTIES OF NORMAL DISTRIBUTION CURVE

- The normal (distribution) curve
  - From $\mu-\sigma$ to $\mu+\sigma$: contains about 68% of the measurements (μ: mean, σ: standard deviation)
  - From $\mu-2\sigma$ to $\mu+2\sigma$: contains about 95% of it
  - From $\mu-3\sigma$ to $\mu+3\sigma$: contains about 99.7% of it

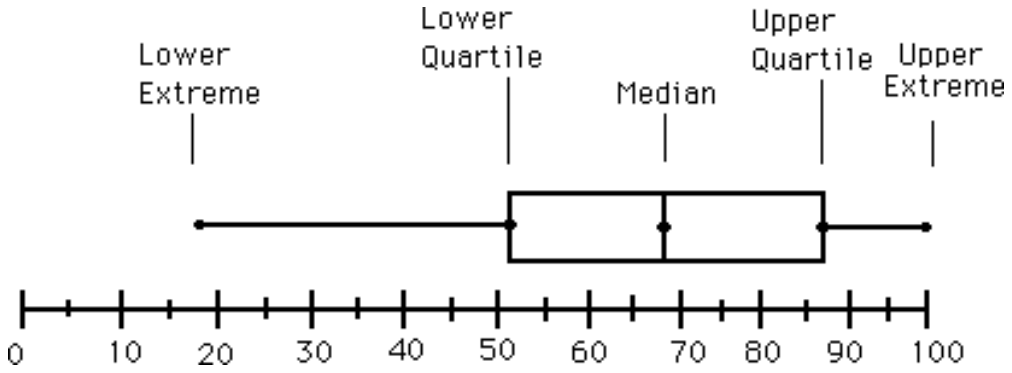


MEASURING DATA DISPERSION/SPREAD/VARIABILITY

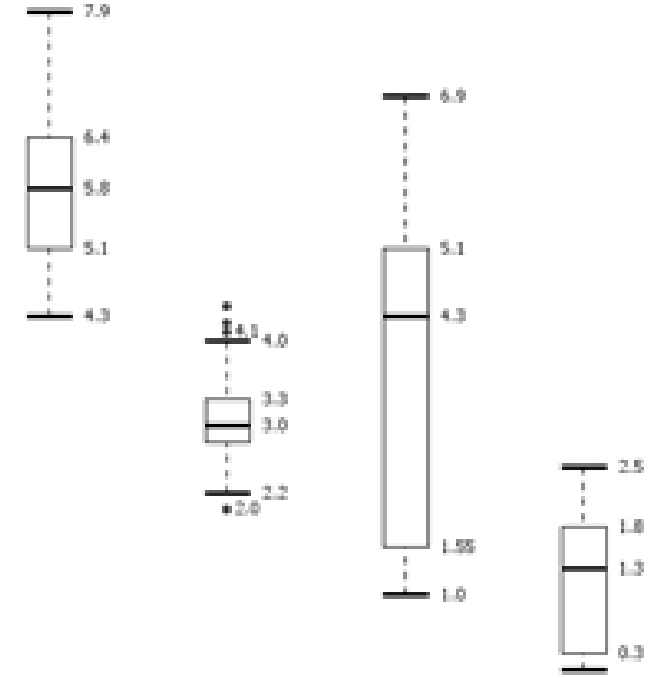
- Quartiles, outliers and boxplots
  - **Quartiles**: Q_1 (25^{th} percentile), Q_3 (75^{th} percentile)
  - **Inter-quartile range**: $IQR = Q_3 - Q_1$
  - **Five number summary**: min, Q_1, median, Q_3, max
  - **Boxplot**: ends of the box are the quartiles; median is marked; add whiskers, and plot outliers individually
  - **Outlier**: usually, a value higher/lower than 1.5 x IQR

BOXPLOT ANALYSIS

- **Five-number summary** of a distribution
  - Minimum, Q1, Median, Q3, Maximum



- **Boxplot**
  - Data is represented with a box
  - The ends of the box are at the first and third quartiles, i.e., the height of the box is IQR
  - The median is marked by a line within the box
  - Whiskers: two lines outside the box extended to Minimum and Maximum
  - Outliers: points beyond a specified outlier threshold, plotted individually



MULTIVARIATE SUMMARY STATISTICS

- Data that consists of several attributes is known as **multivariate data**.
- Spread of each attribute can be computed separately using the techniques discussed already.
- However, for data with continuous variables, spread of the data is most commonly captured by the **Covariance Matrix (S)**, whose ijth entry s_{ij} is the covariance of the ith and jth attributes of the data, i.e., s_{ij} = covariance (x_i, x_j)

$$\begin{array}{c} \\ x \\ y \end{array} \begin{array}{c} \begin{array}{cc} x & y \end{array} \\ \begin{bmatrix} var(x) & cov(x,y) \\ cov(x,y) & var(y) \end{bmatrix} \end{array}$$

$$\begin{array}{c} \\ x \\ y \\ z \end{array} \begin{array}{c} \begin{array}{ccc} x & y & z \end{array} \\ \begin{bmatrix} var(x) & cov(x,y) & cov(x,z) \\ cov(x,y) & var(y) & cov(y,z) \\ cov(x,z) & cov(y,z) & var(z) \end{bmatrix} \end{array}$$

READING MATERIAL

- Data transformation: https://learningds.org/ch/09/wrangling_transformations.html
- Chapter#2: Hands-On Machine Learning, Aurélien Géron